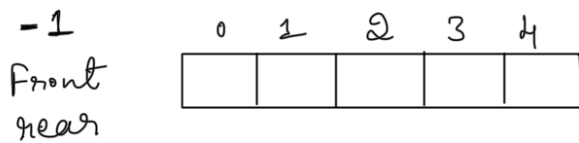


QUEUE PART-2

DAY 52
31/08/2025

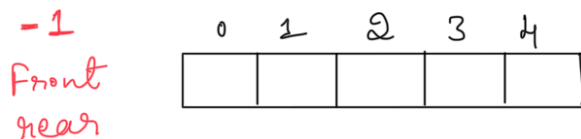
Implementing Queue Using Array.



We Insert elements from Rear End.

Rear will be incremented when element is inserted in queue.

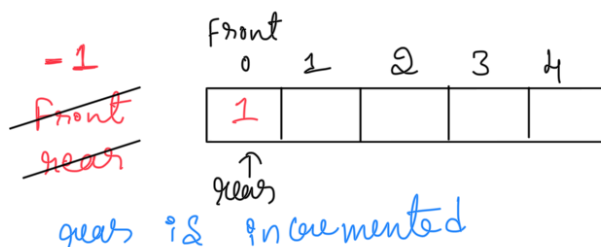
Any operations on Queue
Insert Delete Peek
Have Time Complexity $O(1)$



Initially Front and Rear are pointing to same index -1

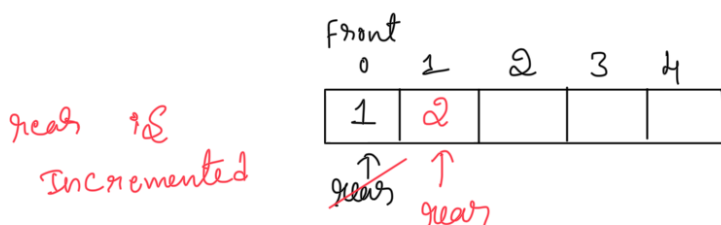
Insertion will happen in Rear side

Insert 1 to Queue (rear & front both incremented)

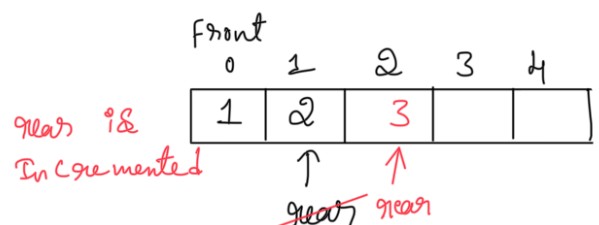


When we Insert a Element to the Empty Queue then rear and front point to same index Bcz Queue has only single Element to point

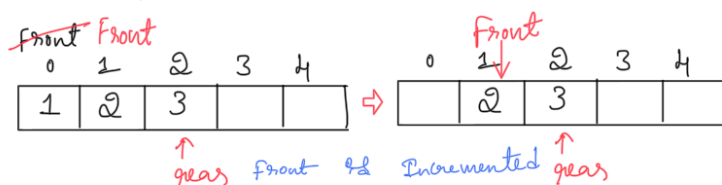
Insert 2 to Queue



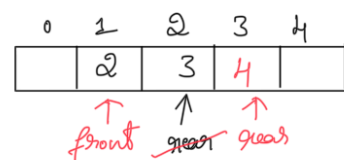
Insert 3 to Queue



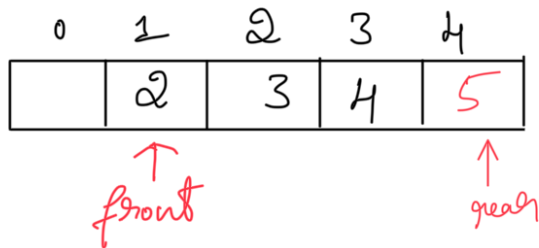
Delete from Queue dequeue



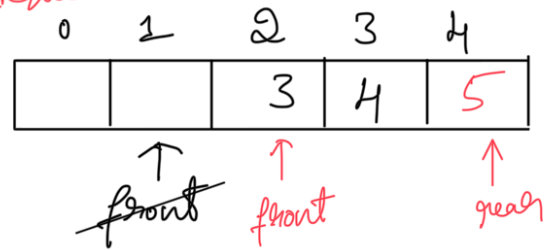
Enque 4



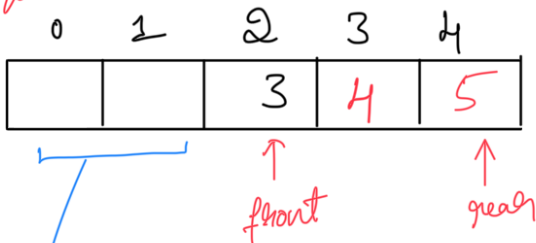
Enque 5



Dequeue



Enque 6



Here we can't add more elements to the queue
 Bcz rear is pointing to last index of the queue
 $(rear == size - 1)$ → capacity of queue

Even though there is a space in the queue we can't add more elements
 Bcz the rear is pointing to last index of queue which means the queue is full. But practically there is some space left out in the queue,

This causes memory waste and this is the drawback of the (Array Linear) Queue Implementation, some amount of space will be wasted in some queue full conditions.

To overcome this issue we use the Circular Array.

The Circular array can be achieved by modulus operator "%".

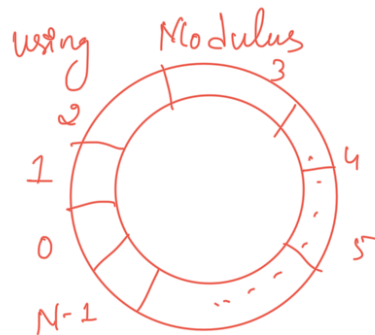
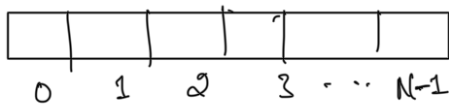
As we know that the array index starts from 0 and ends at size - 1 (which is considered as last index of array)

Speciality of Modulus Operator

Any Number or index value is divided By the Array of Size "N" the remainder will be lie in the range 0 1 2 ... N-1

$$\text{index \% N} = \overset{\text{Remainder}}{0, 1, 2, \dots, N-1}$$

Array of size N



The Modulus operator will not let the index go out of range, It make sure that the index lies in the Valid Range after each updation of index.

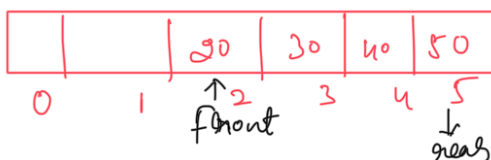
Example if Array size is 5

$$\text{index \% 5} \quad \text{the remainder lies in } 0, 1, 2, 3, 4$$

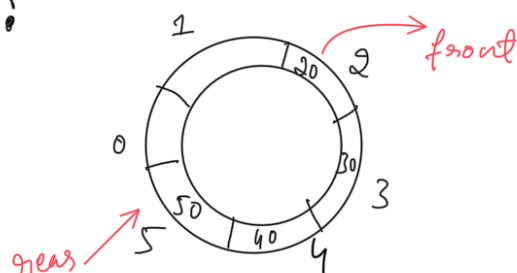
$$\begin{aligned} 1 \% 5 &= 1 & 7 \% 5 &= 2 \\ 2 \% 5 &= 2 \\ 3 \% 5 &= 3 \\ 4 \% 5 &= 4 \\ 5 \% 5 &= 0 \\ 6 \% 5 &= 1 \end{aligned}$$

Modulus operator never let the value go up from range of 0 to N-1

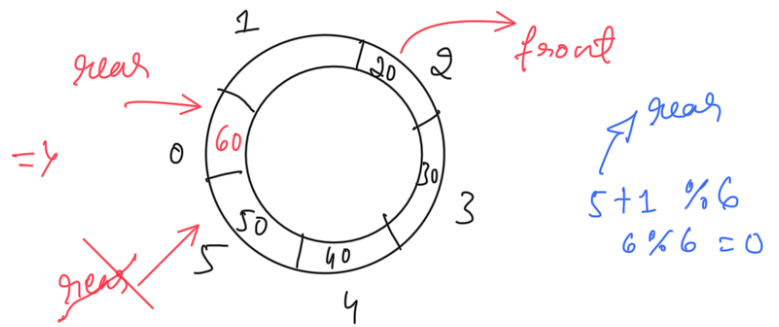
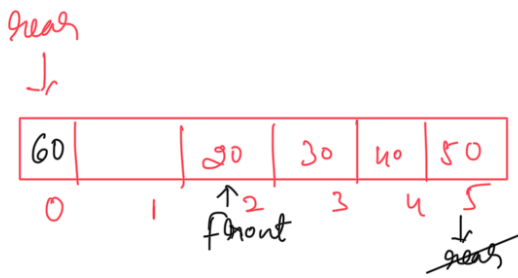
In Circular Queue Empty Spaces are utilize if size-1.
rear is pointing at index



=>



Insert $\Rightarrow$ Engine 60



Insert $\Rightarrow$ Engine 60

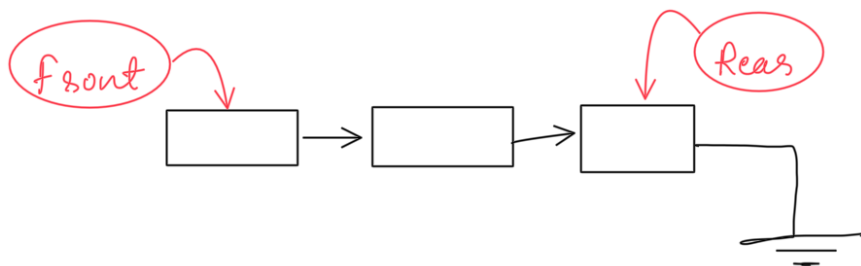
In Every Insertion gear will be incremented as $(\text{gear} + 1) \% \text{size}$

And Queue is Full When $(\text{gear} + 1) \% \text{size} = \text{front}$

In Circular Queue if $(\text{gear} + 1) \% \text{size} = \text{front}$ then that Queue is full.

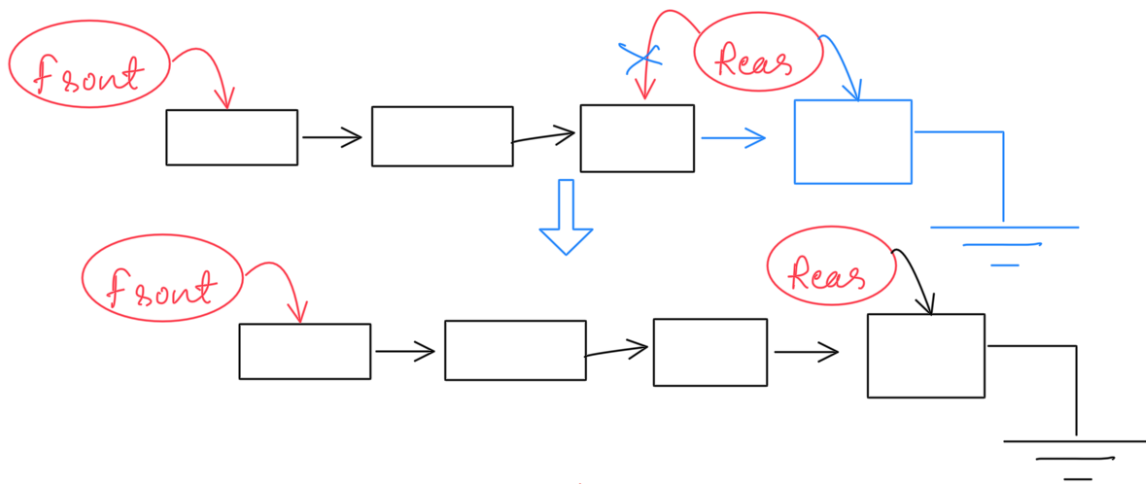
Queue Implementation using Singly linked list.

We maintain two pointers gear and front in the Singly linked list and we use dynamic Memory Allocation (DMA)



Engine Adding Elements to the Queue

We add New Elements at the Rear / End node of linked list

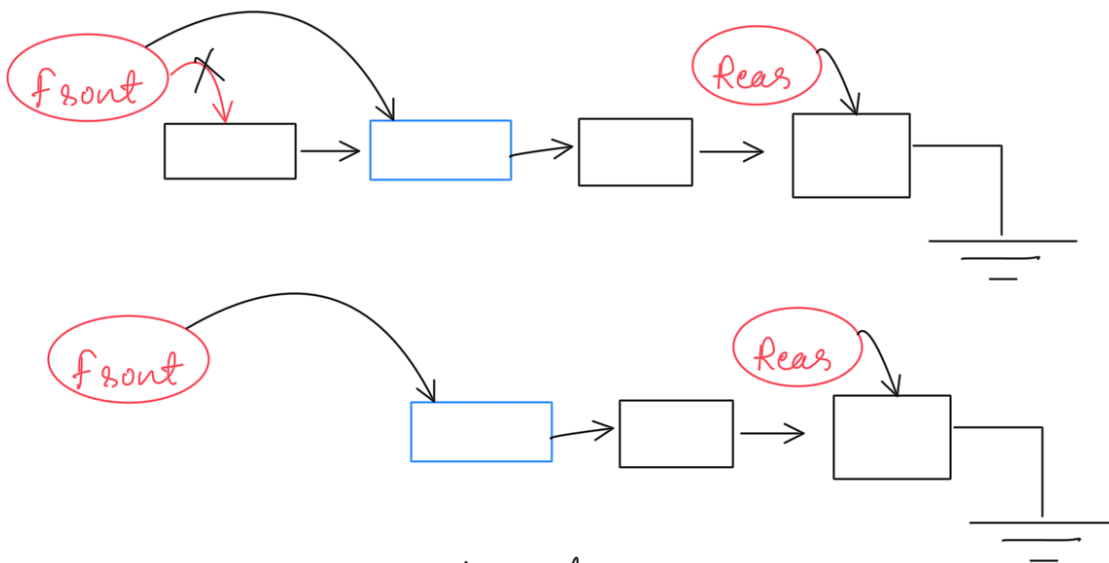


Adding Element
to the Queue
and updating
Rear

Rear.next = newNode
Rear = Rear.next

NOTE: The Queue grows
Dynamically No fixed size
Queue is Empty when Front = Null

Dequeue Elements from the Queue



front = front.next

Corner Cases in Insertion

If Queue is Empty

otherwise front = rear = newNode

rear.next = newNode

rear = newNode

In Deletion

if Front == null
Cannot delete

otherwise

Front = Front.next

Since memory is allocated dynamically in the heap there is no size limit for queue until the heap memory is out of space

Feature	Linear Array Queue	Circular Array Queue	Linked List Queue
Empty Condition	<code>front > rear or front == -1</code>	<code>front == -1</code>	<code>front == null</code>
Full Condition	<code>rear == size-1</code>	<code>(rear+1) % size == front</code>	Only if memory full
Memory	Fixed, may waste	Fixed, efficient use	Dynamic, uses extra pointers
Efficiency	Poor after many deletions	Good (no space waste)	Good, dynamic
Ease of Implementation	Easiest	Medium (needs modulo)	Harder
Best Use Case	Simple, small queue	Bounded, high throughput queue	Large/unbounded queue

✔ Summary:

- Linear Array Queue → only good for very simple, small, temporary queues.
- Circular Array Queue → best when queue size is fixed but you want full space utilization (e.g., CPU scheduling, circular buffers).
- Linked List Queue → best when queue size is unpredictable or large (e.g., dynamic task scheduling).

Tomorrow I will write Java code for these 3 queue implementation methods.